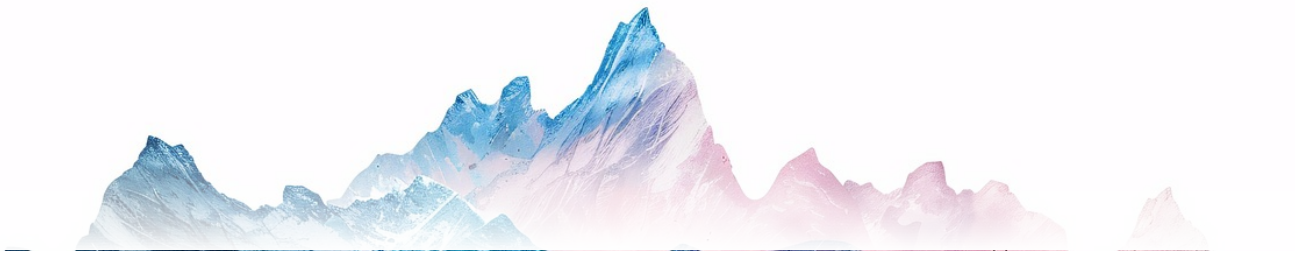
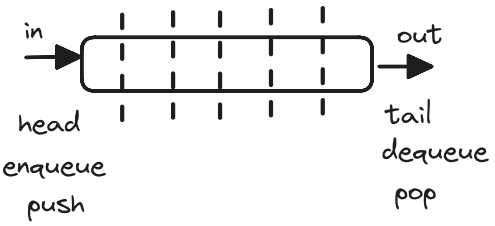


Queue

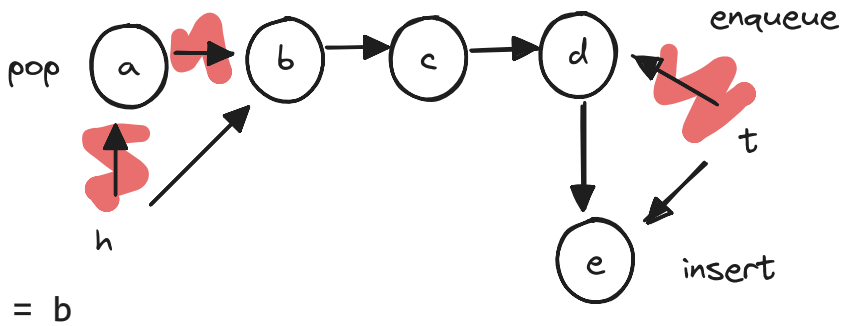
Queue

- FIFO
- singly linked list
- insert at head, pop at tail



---

dequeue




```
h = b
return a;
```

```
t-> = e
t = e
```

```
h = this.head;
this.head = this.head.next;
this.length--;
// free in no gc languages
return h;
```

```
this.tail.next = e;
this.tail = e;
this.length++;
```

Create a node type

```
type Node<T> = {
  value: T;
  next?: Node<T>;
}
```

Initialize queue

```
this.head = this.tail = undefined;
this.length = 0;
```

Object Literal initialization for enqueue

```
{value: T} as Node<T>
```

Node type:
This defines a TypeScript type called `Node<T>`, which represents a generic linked list node.

```
type Node<T> = {
```

```
    value: T;
    next: Node<T>;
}
```

> **The `?` Operator**
> The `?` in TypeScript is called the optional chaining operator. In the expression `this.head?.value()`, it's being used to safely access the `value()` method on `this.head` object.

Without optional chaining, you'd have to write something like:

```
return this.head ? this.head.value() : undefined;
```

> **`!head` vs `head === undefined`**
> `!head` valuates to true if `head` is any falsy value (including undefined, null, 0, "", false, and NaN)

> **Object literal creation in typescript**
>
> ```
> this.head = this.tail = { value: item } as Node;
> ```

1. **Object Literal Creation**
   Creates a new object with a `value` property set to `item`:

   ```
   { value: item }
   ```

2. **Type Assertion**
   Uses the `as` keyword to assert this object conforms to the `Node` type, even if TypeScript's type inference doesn't recognize it as such[1][5][7]. This is necessary because TypeScript might infer the object type as `{ value: T }` rather than `Node`.